

Enumerations

In Java, enum is a **special data type** used to define a **fixed set of constant values** in a **type-safe way**.

It improves code clarity by replacing numeric or string constants with meaningful names.

- Provides type safety by restricting values to predefined constants
- Improves code readability and maintainability
- Commonly used for fixed data like days, directions, and states

Declaration of enum in Java

Enum declaration can be done outside a class or inside a class but not inside a method.

```
enum EnumName {  
    CONSTANT1, CONSTANT2, CONSTANT3;  
}
```

Declaration outside the class

enums can be declared outside a class and accessed directly

```
enum TrafficLight {  
    RED, GREEN, YELLOW;  
}  
public class Test{  
  
    public static void main(String[] args){  
  
        TrafficLight x = TrafficLight.RED;  
        System.out.println(x);  
    }  
}  
RED
```

Declaration inside a class

Enums can also be declared inside a class but not inside a method.

```
public class Test {  
    enum Color {  
        RED,  
        GREEN,  
        BLUE;  
    }  
  
    public static void main(String[] args) {  
        Color c1 = Color.RED;  
    }  
}
```

```
        System.out.println(c1);
    }
}
```

RED

Properties of Enum in Java

There are certain properties followed by Enum as mentioned below:

An enum can have:

- Member variables (instance variables)
- Member functions (methods)
- Constructors
- Abstract methods
- **Class Type:** Internally implemented as a class.
- **Enum Constants:** Each constant is an object of the enum type.
- **Switch Support:** Can be used in switch statements.
- **Implicit Modifiers:** Constants are public static final.
- **Inheritance:** Enums cannot extend classes but can implement interfaces.

Enum in a Switch Statement

```
class Test{
    enum Color {
        RED, // public static final Color RED;
        GREEN,
        BLUE,
        YELLOW;
    }
    public static void main(String[] args){
        Color var = Color.YELLOW;
        switch (var) {
            case RED:
                System.out.println("Red color");
                break;
            case GREEN:
                System.out.println("Green color");
                break;
            case BLUE:
                System.out.println("Blue color");
                break;
            default:
                System.out.println("Other color");
        }
    }
}
```

Other color

Enum with Methods and Constructor

```
enum Color{
    RED, GREEN, BLUE;
    private Color(){
//enum objects should be created only by Java, not by programmers
        System.out.println("Constructor called for: " + this);
    }
    public void display(){
        System.out.println("Color is: " + this);
    }
}

public class Test{
    public static void main(String[] args){
        Color c1 = Color.RED;
        c1.display();
    }
}

Constructor called for: RED
Constructor called for: GREEN
Constructor called for: BLUE
Color is: RED
```

Internally Java does something like:

```
public static final Color RED = new Color();
public static final Color GREEN = new Color();
public static final Color BLUE = new Color();
```

Enum with Abstract Methods

```
//Basic Syntax
enum EnumName {
    CONSTANT1 {
        // implementation of abstract method
    },
    CONSTANT2 {
        // implementation of abstract method
    };
    public abstract returnType methodName();
}
```

```
enum Day{
    MONDAY{
        public String getNumber(){
            return "1st day";
        }
    },
    FRIDAY{
        public String getNumber(){
            return "5th day";
        }
    }
}
```

```

};
    public abstract String getNumber();
}
public class EnumTest {
    public static void main(String[] args){
        System.out.println(Day.MONDAY.getNumber());
        System.out.println(Day.FRIDAY.getNumber());
    }
}
1st day
5th day
enum Operation {
    ADD {
        public int calculate(int a, int b) {
            return a + b;
        }
    },

    SUBTRACT {
        public int calculate(int a, int b) {
            return a - b;
        }
    };
    public abstract int calculate(int a, int b);
}
public class Test {
    public static void main(String[] args) {
        System.out.println(Operation.ADD.calculate(10, 5));
        System.out.println(Operation.SUBTRACT.calculate(10, 5));
    }
}
15
5

```

Enum with Return Method

```

enum TrafficSignal {
    RED,
    GREEN,
    YELLOW;
    public String action() {
        if(this == RED)
            return "STOP";
        else if(this == GREEN)
            return "GO";
        else
            return "WAIT";
    }
}
public class Test {

```

```

public static void main(String[] args) {
    System.out.println(TrafficSignal.RED.action());
    System.out.println(TrafficSignal.GREEN.action());
}
}
STOP
GO

```

Returning Enum from a Method

```

enum Color {
    RED,
    GREEN,
    BLUE
}
public class Test {
    // Method returning enum type
    public static Color getFavoriteColor() {
        return Color.BLUE;
    }
    public static void main(String[] args) {
        Color c = getFavoriteColor();
        System.out.println(c);
    }
}
BLUE

```

Enum and EnumSet (Specific Range Iteration)

EnumSet: a Set implementation built **only for enum types**, from java.util.

EnumSet.range() allows iteration over a specific range of enum constants

```

import java.util.EnumSet;
public class Test{
    enum Day{
        SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY,
        SATURDAY;
    }
    public static void main(String[] args){
        EnumSet<Day> workDays = EnumSet.range(Day.TUESDAY, Day.FRIDAY);
        for (Day d : workDays){
            System.out.println(d);
        }
    }
}
TUESDAY
WEDNESDAY
THURSDAY
FRIDAY

```

Enum can implement interfaces

```
interface Printable {
    void print();
}
enum Color implements Printable {
    RED,
    GREEN;
    public void print() {
        System.out.println(this);
    }
}
public class Test {
    public static void main(String[] args) {
        Color.RED.print(); //Color temp = Color.RED; temp.print();
    }
}
RED
```

Enum as a Member Variable of a Class

```
enum Gender {
    MALE,
    FEMALE,
    OTHER
}
class Student {
    String name;
    int age;
    Gender gender; // Enum member variable
    Student(String name, int age, Gender gender) {
        this.name = name;
        this.age = age;
        this.gender = gender;
    }
    void display() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Gender: " + gender);
    }
}
public class Test {
    public static void main(String[] args) {
        Student s1 = new Student("MSD",40,Gender.MALE);
        s1.display();
    }
}
Name: MSD
Age: 40
Gender: MALE
```

Method	Description
values()	Returns all enum constants as an array
valueOf(String)	Converts a string to an enum constant
ordinal()	Returns the index of the constant (0-based)
name()	Returns the constant name as a String
compareTo()	Compares the order of enum constants

```
enum Color {
    RED,
    GREEN,
    BLUE
}

public class EnumMethodsDemo {

    public static void main(String[] args) {

        // 1. values() - Returns all enum constants as an array
        System.out.println("Using values():");

        for (Color c : Color.values()) {
            System.out.println(c);
        }

        // 2. valueOf(String) - Converts String into enum constant
        System.out.println("\nUsing valueOf():");

        Color c1 = Color.valueOf("GREEN");
        System.out.println("Selected Color: " + c1);

        // 3. ordinal() - Returns index position (starts from 0)
        System.out.println("\nUsing ordinal():");

        System.out.println("RED position: " + Color.RED.ordinal());
        System.out.println("GREEN position: " + Color.GREEN.ordinal());
        System.out.println("BLUE position: " + Color.BLUE.ordinal());

        // 4. name() - Returns enum constant name as String
        System.out.println("\nUsing name():");

        String colorName = Color.BLUE.name();

        System.out.println("Name of constant: " + colorName);

        // 5. compareTo() - Compares order of enum constants
        System.out.println("\nUsing compareTo():");
```

```
System.out.println(
    "RED compared with GREEN: "
    + Color.RED.compareTo(Color.GREEN)
);

System.out.println(
    "BLUE compared with GREEN: "
    + Color.BLUE.compareTo(Color.GREEN)
);

System.out.println(
    "GREEN compared with GREEN: "
    + Color.GREEN.compareTo(Color.GREEN)
);
}
```

Using values():

RED

GREEN

BLUE

Using valueOf():

Selected Color: GREEN

Using ordinal():

RED position: 0

GREEN position: 1

BLUE position: 2

Using name():

Name of constant: BLUE

Using compareTo():

RED compared with GREEN: -1

BLUE compared with GREEN: 1

GREEN compared with GREEN: 0